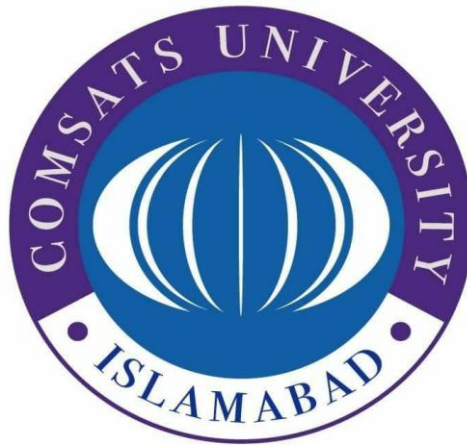




BS-ARTIFICIAL INTELLIGENCE OBJECT ORIENTED PROGRAMMING

SUBMITTED TO: Dr Muhammad Shahid Bhatti

SUBMITTED BY: MARYAM ASHFAQ

ROLL NO. 030

ID: SP26-BAI-030

SUBMISSION DATE: 25/09/2026

THEORY ASSIGNMENT 1

Object Oriented Analysis of a Smart Campus Allocation System

Task 1 — Stakeholder and Requirement Analysis

1.1 Stakeholders and their primary needs

Stakeholder	Primary need
Student	Fair, predictable access to classrooms, labs, seminar rooms and project spaces for individual and group academic work, without having to reveal personal details to other students.
Faculty member	Priority for scheduled teaching sessions and the ability to make urgent academic bookings that ordinary student requests cannot block.
Administrator	High utilization of learning spaces and reliable reporting, to plan capacity and account to the university for how space is used.
Maintenance staff	The authority to immediately block a room that becomes unsafe — even one with confirmed reservations — without waiting for administrative sign-off.
Security staff	A traceable, timestamped record of who accessed which space and when, to investigate incidents and control building entry.
Disability support staff	Assurance that accessible spaces remain identifiable and available for the users who need them.

Requirement conflicts

1. Fair access vs. faculty priority. Students expect fair access, while faculty may need priority for scheduled teaching and urgent academic work. A transparent priority policy can protect scheduled teaching while applying a documented fair-ordering rule to remaining requests.

2. Utilization vs. safety. Administration benefits from high utilization, but maintenance must be able to block unsafe rooms immediately. Safety status therefore overrides normal allocation and triggers cancellation or relocation handling where necessary.

3. Traceability vs. controlled information access. Security requires records, while not every user needs administrative details. Detailed audit information is therefore restricted to authorized roles.

Facts and assumptions

The brief establishes multiple stakeholder groups, request types, changing roles, room blocking, audit records, notifications, and emergency cancellation/relocation. Assumptions: no overlapping confirmed bookings, deterministic handling of simultaneous requests, safety blocks take priority, and reservation history is retained for audit.

Task 2 — Object Oriented Conceptual Model

2.1 Proposed classes

Ten classes are proposed; Classroom and Laboratory are concrete subclasses of the abstract LearningSpace, which justifies the inheritance relationship shown in the diagram below.

Class	Responsibility	Key attributes / operations
User	Represents any person using the system and holds their identity and session.	userId; name (hidden); contactInfo (hidden) — authenticate(); viewAvailability(); requestReservation()
Role	Represents one assignment of a role type (student, faculty, admin, security, maintenance, disability support) to a User, with its own validity window and permissions.	roleType; validFrom/validUntil; permissionSet — hasPermission(action); isActive()
LearningSpace (abstract)	Represents a physical space and its own availability/safety status, independent of who books it.	spaceId; capacity; status; isAccessible — checkAvailability(slot); block(reason); release()
Classroom (extends LearningSpace)	A teaching space with classroom-specific rules and features.	boardType — approveForClassSession()
Laboratory (extends LearningSpace)	A specialised space with safety and equipment rules a plain classroom does not have.	labType; safetyRequirements — approveForLabSession()
Reservation	Represents one claim on one space for one time slot on behalf of one requester, and owns its own life cycle.	reservationId; purpose (hidden); timeSlot; status — confirm(); cancel(); relocate(newSpace)
ApprovalPolicy	Owns the business rule for evaluating a request and resolving conflicts between competing requests — kept out of Reservation and User.	activityType; priorityRules — evaluate(request); resolveConflict(a, b)
Equipment	Represents a piece of movable or fixed equipment that can be allocated to a reservation, separately from the room itself.	equipmentId; type; isMovable — reserveWith(reservation); checkAvailable()
NotificationService	Owns all outgoing communication to users, so no other class needs to know how messages are sent.	channelType; templates — notify(user, message); broadcastToAffected(event)
AuditLog	Owns the permanent, append-only history of actions, for security traceability and emergency audits.	logId; entries — record(action); generateAuditTrail()

2.2 UML class diagram

The diagram shows associations with multiplicities, an inheritance pair (Classroom and Laboratory specialise the abstract LearningSpace), a composition (a Role assignment has no meaning without the User who holds it, so its life cycle is owned by User), and an aggregation (Equipment exists and can be reused independently of any one Reservation).

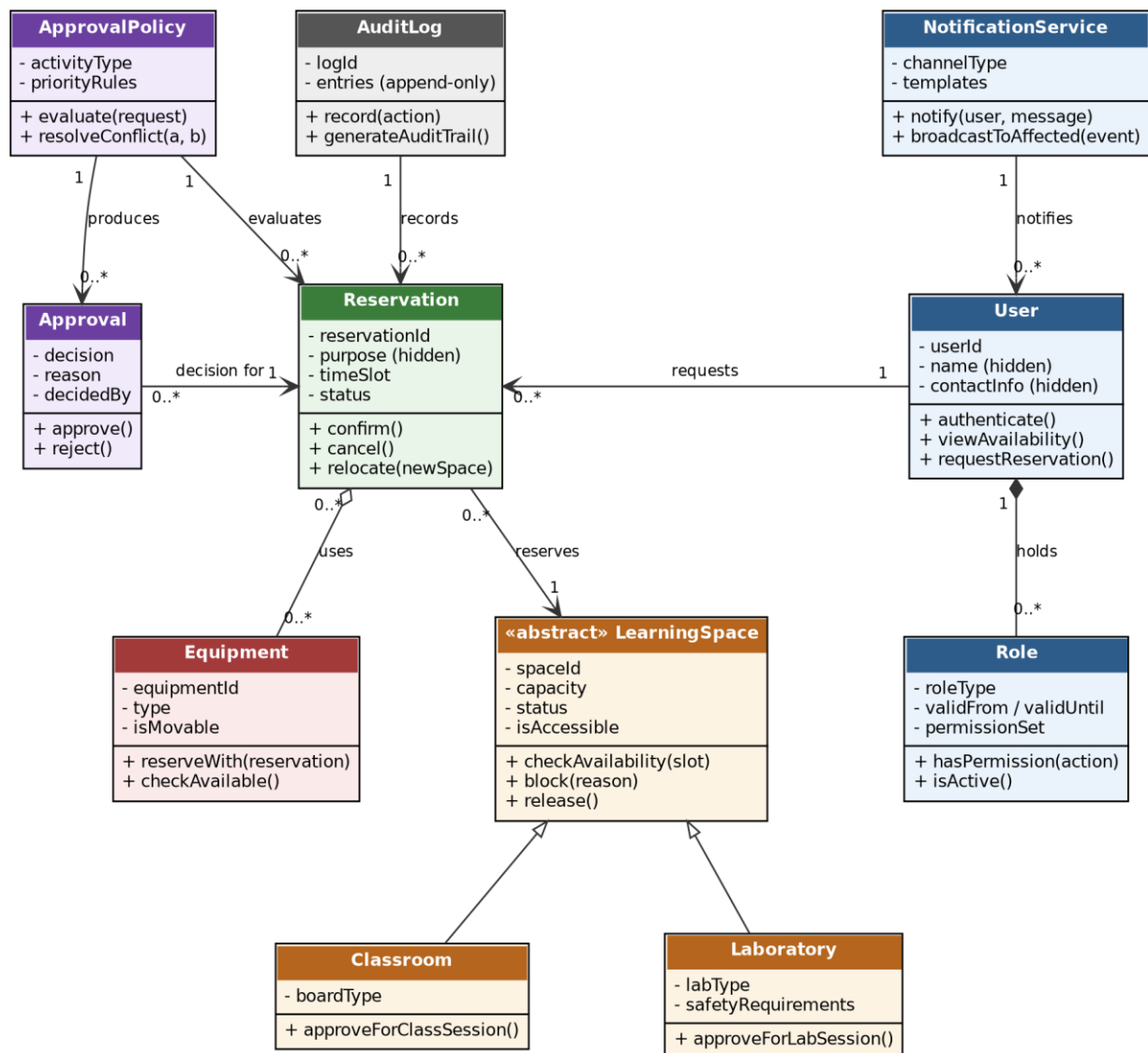


Figure 1 — UML class diagram of the Smart Campus Learning Space Allocation System

2.3 How responsibility is separated

Each class owns exactly one concern (single-responsibility):

- **User** only manages identity, authentication and roles held — it never decides bookings.
- **ApprovalPolicy** owns request evaluation/conflict resolution, so the rule can change independently.
- **LearningSpace** is sole authority over its own safety/availability status.
- **Reservation** owns only booking state and its life-cycle transitions.
- **NotificationService** owns communication; **AuditLog** owns permanent history, so cancelling/relocating can never quietly erase what happened.

Task 3 — Abstraction, Encapsulation and Invariants

3.1 What each selected class abstracts:

- **User** — abstracts a real person interacting with the system, regardless of how many roles they currently hold or how those roles change over the semester.
- **Reservation** — abstracts a specific claim on a specific space for a specific time window, on behalf of a specific requester, independent of how that claim is displayed to other users.
- **LearningSpace** — abstracts the physical resource itself and its current safety/availability status, independent of any particular booking.

3.2 Hidden information and controlled change

Class	Hidden information	Only changed through
User	name and contactInfo (raw personal details); the detailed permission set of each Role.	authenticate() to confirm identity; role changes only through administrator-only operations such as assignRole()/revokeRole(), never by editing fields directly.
Reservation	requestedBy (identity) and purpose, hidden from ordinary users under the Privacy-first condition; only status and timeSlot are exposed publicly.	confirm(), cancel(), relocate(newSpace) — every transition is a guarded method that checks the current state first; status can never be set directly.
LearningSpace	the internal status field and any maintenance notes.	block(reason), restricted to the maintenance role, and internal updates driven by confirmed Reservation objects — never edited directly by a plain User.

3.3 Invariants that must always hold:

- Every Reservation references exactly one LearningSpace and exactly one requesting User (constructor requires both, matching the 1 multiplicities in Figure 1).
- A LearningSpace can never hold two Confirmed reservations with overlapping time slots — checked inside confirm() before state changes.

- A Role's validUntil is never earlier than validFrom, and an inactive Role grants no permission — enforced in isActive(), always checked by hasPermission().
- Once a Reservation leaves Requested/UnderReview, its identity and purpose become read-only except to Security/Administrator roles.
- A cancelled, rejected, or relocated Reservation is never deleted — only its status changes, and AuditLog retains full history; no delete() operation is exposed.

Encapsulation makes these invariants reliable: because fields are private, the only way to change them is through a method that checks current state and caller permission first — so an invalid transition is rejected at the one place the rule lives, rather than depending on every caller to remember it.

Task 4 — Alternative Designs and Decision

4.1 Two ways to represent users with multiple, changing roles

- **Option A — Inheritance.** A base Person class with subclasses Student, Faculty, Administrator, SecurityStaff, MaintenanceStaff, DisabilitySupportStaff, each hard-coding its own permissions.
- **Option B — Composition / role objects (used in Task 2).** A single User class holds a collection of Role instances; each Role carries its own type, validity window and permission set.

4.2 Comparison:

Criterion	Option A — Inheritance	Option B — Composition / role objects
Extensibility	Poor — a new role combination (e.g. a student who is also a teaching assistant) needs a brand-new subclass, or multiple inheritance, which Java does not support for classes.	Good — a new role is simply a new Role instance/type; User itself never changes.
Coupling	High — behaviour is compiled into the class hierarchy, so changing one role's rule can force changes in subclasses or a shared ancestor.	Low — User depends only on the generic Role interface, not on any specific concrete role.
Duplicate behaviour	Likely — shared logic between related roles must be copied or awkwardly pulled into a common ancestor.	Minimal — shared logic lives once, inside Role or a shared policy object that every Role instance reuses.
Multiple simultaneous roles	Not naturally supported — under single inheritance a person is only ever one class at a time.	Naturally supported — User simply holds a set of Role objects, matching the stated requirement that a person may hold more than one role.
Ease of changing permissions at runtime	Poor — changing a person's role effectively means replacing the object with a different subclass instance.	Good — a Role instance can be added, removed, or allowed to expire (validUntil) without touching the User object at all.

4.3 Chosen design and justification:

This model uses Option B. The scenario states that a person may hold more than one role at once and that roles/permissions may change mid-semester. Inheritance cannot represent simultaneous multiple roles without violating single-parent identity, and Java classes do not support multiple inheritance. Role objects satisfy this directly: a User's rights are the union of permissions of every active Role it holds, and a role change is simply adding, removing, or expiring one Role instance rather than replacing the User.

Task 5 — Behaviour and Edge Cases

5.1 Reservation life cycle

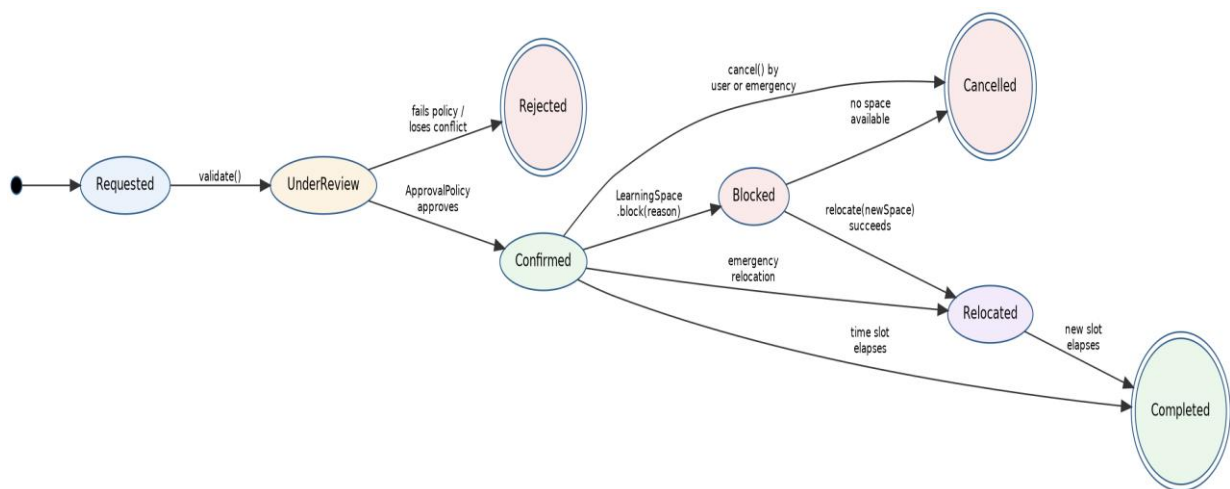


Figure 2 — State diagram for the life cycle of a Reservation

5.2 Handling the three required edge cases

Case 1 — Two simultaneous requests for the same space: `ApprovalPolicy.resolveConflict()` compares requests by priority tier, then by validated-submission timestamp within the same tier. The losing request becomes **Rejected**; `NotificationService` informs that user and `AuditLog` records the decision. *Enforcing object:* `ApprovalPolicy` — the single class that owns the conflict rule.

Case 2 — A maintenance block after confirmation: Maintenance staff call `LearningSpace.block(reason)`. `LearningSpace` transitions every overlapping **Confirmed** reservation to **Blocked** and `NotificationService` notifies affected users; if no equivalent space exists, the reservation moves to **Cancelled**. *Enforcing object:* `LearningSpace` — sole authority over its own safety status.

Case 3 — An emergency cancellation or relocation: An emergency event is raised against one or more `LearningSpace` objects. Every **Confirmed** reservation for the affected spaces is relocated to an equivalent-capacity space or cancelled if none exists. `AuditLog` stores the original assignment and reason; `NotificationService` informs affected users. *Enforcing object:* `Reservation.relocate()/cancel()`, coordinated by `ApprovalPolicy`, with `AuditLog` and `NotificationService` — so the process can never silently overwrite booking state without a record and a notification

Task 6 — Independent Exploration

Role-Based Access Control (RBAC):

RBAC is an authorization approach in which permissions are associated with roles, and users receive permissions through role membership. NIST describes RBAC as simplifying authorization management by managing role permissions and role membership rather than a separate permission structure per user [1], and its model also includes role hierarchies and constraints [2].

Application to the Smart Campus model:

RBAC fits the User and Role classes: one person can hold more than one responsibility simultaneously, and permissions can change during the semester without changing the User object. A student role allows availability viewing and request submission; an authorized security role allows access to traceable records. OWASP guidance also emphasizes least privilege — users receive only the permissions needed for their tasks [3].

- User stores role membership; Role stores permissions.
- Protected operations check the current role before execution.
- Role changes do not require changing the User class.
- Audit information remains accessible to authorized roles only.

Authorization flow:

Request → identify user → load current roles → check permission → validate business rule → perform operation → record audit event → notify affected users. A successful permission check alone does not guarantee a reservation can be confirmed — capacity, status and time-conflict checks still apply.

Example role-permission matrix:

Capability	Student	Faculty	Administration	Security	Maintenance
View availability	Yes	Yes	Yes	Yes	Yes
Submit request	Yes	Yes	As authorized	No	No
Approve request	No	As authorized	Yes	No	No
Block space	No	No	As authorized	No	Yes
View audit records	No	No	As authorized	Yes	As authorized
Relocate/cancel emergency booking	No	As authorized	Yes	As authorized	As authorized

Implementation considerations:

A practical implementation should centralize authorization checks, keep state transitions explicit, record important events in an append-only audit structure, and ensure that allocation checks are atomic so two simultaneous requests cannot both bypass the no-overlap rule. Role membership should be changeable without recreating a Person object.

Generative AI disclosure:

Generative AI assistance was used for drafting and formatting. The final submission remains the student's responsibility, including review of the conceptual model, assumptions, citations and design decisions.

References for this section:

- R. Sandhu, D. F. Ferraiolo and D. R. Kuhn, "The NIST Model for Role-Based Access Control: Towards a Unified Standard," National Institute of Standards and Technology, 2000. Available: <https://www.nist.gov/publications/nist-model-role-based-access-control-towards-unified-standard>
- National Institute of Standards and Technology, "Role Based Access Control," NIST Computer Security Resource Center project page. Available: <https://csrc.nist.gov/projects/role-based-access-control>

References:

- Association for Computing Machinery and IEEE Computer Society, Computing Curricula 2020: Paradigms for Global Computing Education, 2020. Available: <https://www.acm.org/education/curricula-recommendations>
- Seoul Accord, Section D Graduate Attributes, especially Section D.4.1 Range of Problem Solving. Available: <https://www.seoulaccord.org/document.php?id=79>
- R. Sandhu, D. F. Ferraiolo and D. R. Kuhn, "The NIST Model for Role-Based Access Control: Towards a Unified Standard," National Institute of Standards and Technology, 2000. Available: <https://www.nist.gov/publications/nist-model-role-based-access-control-towards-unified-standard>
- National Institute of Standards and Technology, "Role Based Access Control," NIST Computer Security Resource Center project page. Available: <https://csrc.nist.gov/projects/role-based-access-control>

Assignment alignment:

This submission follows the required Tasks 1–6 structure and includes stakeholder analysis, a conceptual class model, abstraction and encapsulation, invariants, alternative user-role designs, a reservation lifecycle, edge-case handling, and an independent exploration of role-based access control. The additional pages expand the behavioural and implementation explanations so the answers are presented with more complete reasoning and readable diagrams.

Conclusion:

The proposed design uses clear object responsibilities and controlled state changes to support a maintainable Smart Campus allocation system. Composition with Role objects provides flexibility for changing responsibilities, while Space and Reservation enforce allocation and lifecycle rules. Explicit auditing, authorization checks and event notifications make exceptional situations traceable and manageable.